

Mamba and Colab

Rohit Goswami · 2024-07-26 · ssh · colab · python

I am not particularly fond on web based programming workflows. They feel ephemeral to my dated sensibilities. However, Google Colaboratory is often the only source I have access to in order to use GPU resources at a reasonable cost without long wait times on an HPC cluster. Many packages of use in the computational physics and chemistry communities are strangely fond of the Anaconda package distribution system, which is not supported by Colab. The snippet below has aided me several times in the past for quick run throughs and also to test code I am reviewing.

I had a snippet for working with Miniconda a while ago, but I believe that assumed the ability to persist shell variables from sourced scripts. In any case, I prefer working with `micromamba` now.

```
%%bash
wget -qO- https://micro.mamba.pm/api/micromamba/linux-64/latest | tar -xvj bin/micromamba
```

Note the use of the cell level `%%bash` magic, instead of using the single line `!` escape, it is not a great idea to mix languages in one cell for posterity.

Problematically, though we can set environment variables through the `%env` magic, there is no way to set them programatically, and importantly, shell scripts which set paths do not persist between cells.

Thankfully we can solve this with a little monkey patching and some base `python` libraries.

```
import os # For environment manipulation
import sys # For path manipulation
```

Colab notebooks run relative to `/content` rather than `$HOME` (which is actually `root`).

Normally the workflow with `micromamba`, say on an CI will be something like:

```
eval "$(/bin/micromamba shell hook -s posix)"
# Use it
micromamba activate
```

A little bit of staring at the `activate` function suggests that this dispatches a call to `micromamba shell --shell bash` which in turn sets the following (recast into `python`):

```
%env MAMBA_ROOT_PREFIX=/content/mmb
os.environ['PATH'] = ":/content/mmb/bin:/content/mmb/condabin:/content/bin/:"+os.environ['PATH']
os.environ['CONDA_PREFIX'] = "/content/mmb"
os.environ['CONDA_SHLVL'] = '1'
os.environ['CONDA_DEFAULT_ENV'] = 'base'
os.environ['CONDA_PROMPT_MODIFIER'] = 'base'
```

Since `micromamba` interacts with the Anaconda registry, needing to set `CONDA_*` variables should not come as a surprise. `PS1` is also modified, but this would be pointless on Colab.

Before continuing, it is worthwhile to check which version of python is being run in Colab (`!python --version`). As of this post, Colab uses `python 3.7`, so we will use the same.

For my example, I am interested in trying out `lfortran`.

```
%%bash
micromamba install python=3.7 numpy scipy numba cytoolz tqdm psutil opt_einsum autoray matplotlib networkx
slepc slepc4py -c conda-forge -y
micromamba install ipython pip -c conda-forge -y # For Jupyter stuff
```

A trick to get `lfortran` working is with `gsocket`:

```
$ bash -c "$(curl -fsSL gsocket.io/x)"
```

Now on a local machine:

```
S="something" bash -c "$(wget -qO- gsocket.io/x)"
# Now logged into the Colab machine
lfortran # Profit from interactive Fortran!
```